

Zoo Network: Decentralized AI Training and Inference Layer with Semantic Optimization

Zach Kelling^{*1}

¹*Hanzo Industries Lux Industries Zoo Labs Foundation,*
`research@lux.network`

Version v2025.09 – September 2025

Abstract

We present **Zoo Network**, a specialized Layer 2 blockchain infrastructure for decentralized artificial intelligence training, inference, and governance. Built atop the Hanzo compute layer, Zoo extends base blockchain capabilities with AI/ML-specific primitives including the Hamiltonian Large Language Model (HLLM) framework, Training-Free Group Relative Policy Optimization (GRPO), and the Experience Ledger—a content-addressable, cryptographically verifiable repository of semantic optimization knowledge. Unlike centralized AI platforms, Zoo enables community-driven model evolution through DAO governance, privacy-preserving federated learning, and a novel economic model where data contributors earn inference credits and governance rights. By operating in the context space rather than parameter space, Zoo achieves 99.8% cost reduction compared to traditional fine-tuning while maintaining transparency and auditability. We demonstrate Zoo’s architecture across storage (IPFS/Arweave), computation (Hanzo GPU nodes), and consensus (Lux multi-consensus base), providing a complete stack for decentralized AI infrastructure. This work represents a fundamental shift from corporate-controlled AI toward community-owned, verifiable, and composable intelligence systems.

1 Introduction

The contemporary artificial intelligence landscape is characterized by extreme centralization. A handful of corporations control foundation models, training infrastructure, and inference APIs, creating critical dependencies

^{*}zach@lux.network

and single points of failure [3]. Users contribute data but receive no ownership, researchers cannot audit model behavior, and improvements remain locked behind proprietary walls.

1.1 Motivation

Three fundamental problems plague centralized AI:

1. **Opacity:** Model weights, training data, and fine-tuning methodologies are proprietary black boxes. Even when APIs are available, the underlying system state is unknowable.
2. **Lock-in:** Users who contribute feedback improve models owned by corporations. There is no portability, interoperability, or long-term access guarantee.
3. **Cost Barriers:** Fine-tuning a 32B parameter model costs \$10,000+ and requires specialized infrastructure [35]. This excludes academic researchers, non-profits, and developing nations.

1.2 Our Contribution

Zoo Network addresses these challenges through:

- **Layered Architecture:** Building on Lux (L0 consensus) and Hanzo (L1 compute), Zoo provides an AI-specialized L2 with smart contract coordination and decentralized storage.
- **Training-Free Optimization:** Via HLLM and Training-Free GRPO, models improve by curating semantic experiences in context space—not updating billions of parameters. Cost drops from \$10K to \$18 per task [35].
- **Experience Ledger:** A Merkle-tree-verified, content-addressable repository of human-readable optimization insights. All changes are auditable, governable, and composable.
- **Economic Incentives:** Contributors earn inference credits, governance votes, and revenue shares proportional to their data/experience contributions.
- **Privacy Preservation:** Federated learning and optional zero-knowledge proofs enable private model training without exposing sensitive data.

This paper is organized as follows: Section 2 surveys related work. Section 3 details Zoo’s layered architecture. Section 4 explains HLLM and Training-Free GRPO. Section 5 describes the Experience Ledger. Sections 6-8 cover training, inference, and economics. Section 9 addresses security. Section 10 compares Zoo to centralized platforms. Section 11 concludes with future work.

2 Related Work

2.1 Decentralized Machine Learning

Prior efforts toward decentralized ML include:

- **Federated Learning (FL)** [24]: Trains models across devices without centralizing data. However, coordination typically remains centralized (e.g., Google’s FL infrastructure).
- **Blockchain-based FL** [20]: Uses blockchain for coordination but lacks AI-specific primitives (experience management, context optimization).
- **Ocean Protocol / SingularityNet** [25,34]: Focus on data marketplaces and AI service trading but do not address training optimization or semantic knowledge curation.
- **Bittensor** [7]: Incentivizes model training via proof-of-intelligence. Zoo complements this by adding semantic experiences and frozen-weight optimization.

2.2 Efficient Fine-Tuning

- **LoRA/QLoRA** [10,16]: Reduce fine-tuning costs via low-rank adaptation. Still requires gradient updates and GPU clusters.
- **RLHF** [26]: Reinforcement learning from human feedback. PPO-based approaches are sample-inefficient and computationally expensive.
- **DPO/IPO** [28]: Direct preference optimization bypasses reward models but still updates parameters.
- **Training-Free GRPO** [35]: Our work builds directly on this, extending it with decentralized governance and cryptographic verification.

2.3 Blockchain AI Infrastructure

- **Akash/Render Network** [1]: Decentralized compute marketplaces but lack AI-specific orchestration.
- **Hanzo Network** [14]: Provides the L1 compute layer on which Zoo is built. Hanzo handles consensus, GPU allocation, and base infrastructure.
- **Lux Blockchain** [22]: L0 multi-consensus layer with post-quantum cryptography. Zoo inherits security from Lux.

Zoo uniquely combines semantic optimization, cryptographic auditability, and economic incentives into a cohesive L2 stack.

3 Layered Architecture

Zoo Network operates as a specialized Layer 2 atop Hanzo’s base compute infrastructure. This section details the multi-layer design.

3.1 Layer 0: Lux Consensus

Lux [22] provides the foundational consensus layer with:

- **Wave Consensus:** DAG-based finality via repeated sub-sampled voting [31]. Achieves sub-second confirmation with Byzantine fault tolerance.
- **Post-Quantum Cryptography:** CRYSTALS-Dilithium signatures resist quantum attacks.
- **Multi-Chain Architecture:** Supports heterogeneous virtual machines (EVM, WASM, custom).

Zoo inherits security and finality guarantees from Lux’s validator set.

3.2 Layer 1: Hanzo Compute Infrastructure

Hanzo [14] extends Lux with compute-specific capabilities:

- **GPU Compute Nodes:** Distributed GPU clusters run containerized LLM inference. Nodes stake collateral and earn rewards proportional to compute provided.
- **Proof of Compute:** Validators verify inference correctness via sampling or zero-knowledge proofs.
- **Storage Primitives:** Integration with IPFS for content-addressable data and Arweave for permanent archival.
- **Rust Crates:** Hanzo provides reusable libraries (consensus, mining, HTTP APIs, libp2p networking) that Zoo extends.

Hanzo is blockchain-agnostic compute infrastructure; Zoo specializes it for AI/ML workloads.

3.3 Layer 2: Zoo AI/ML Specialization

Zoo adds AI-specific components:

1. Smart Contracts:

- *Inference Router:* Routes queries to available GPU nodes, specifies experience library versions, records proofs.

- *Experience Registry*: Stores Merkle roots of experience libraries, handles DAO votes for curation.
- *Governance Contract*: Manages proposals, voting, and treasury allocation.

2. Off-Chain Components:

- *Semantic Context Manager*: Maintains experience embeddings, performs similarity search for retrieval.
- *Experience Storage*: IPFS (current library), Arweave (immutable history).
- *GPU Compute Nodes*: Load frozen base models, inject experiences into context windows, execute inference.

3. Rust Libraries (extend Hanzo):

- `zoo_mcp`: Model Context Protocol for experience management.
- `zoo_embedding`: Vector embedding generation.
- `zoo_job_queue_manager`: Training job scheduling.
- `zoo_tools_primitives`: AI-specific utilities.

3.4 Data Flow

1. User submits query via Inference Router contract.
2. Router selects GPU node and experience library version.
3. GPU node fetches base model (frozen weights) and experience library (IPFS).
4. Experiences are injected into context window.
5. Model generates response.
6. Response returned to user; proof recorded on-chain.

This architecture separates concerns: Lux handles consensus, Hanzo handles compute, Zoo handles AI-specific logic.

4 Hamiltonian Large Language Models (HLLM)

4.1 Core Concept

Traditional fine-tuning modifies model parameters θ via gradient descent:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta, \mathcal{D})$$

This is expensive (requires backpropagation through billions of parameters) and opaque (weight changes are inscrutable).

HLLM [40] proposes an alternative: keep θ frozen and optimize the *context* Ψ instead. The system obeys a **Hamiltonian invariant**:

$$\Psi \cdot \Theta = \kappa$$

where:

- Ψ = Policy mass (semantic context / experiences)
- Θ = Inference cost (model entropy / uncertainty)
- κ = Conserved constant (system equilibrium)

As context expands ($\Psi \uparrow$), model uncertainty decreases ($\Theta \downarrow$) while preserving total system cost κ .

4.2 Training-Free GRPO

Group Relative Policy Optimization (GRPO) [32] computes advantages relative to group means instead of baselines:

$$A_i = \frac{R_i - \mu_G}{\sigma_G}$$

where R_i is the reward for output i , μ_G is the group mean, and σ_G is group standard deviation.

Training-Free GRPO [35] replaces numerical advantages with *semantic advantages*—natural language insights extracted via LLM introspection. The algorithm proceeds in three stages:

4.2.1 Stage 1: Trajectory Summarization

For each generated output o_i , an LLM summarizes:

- What actions were taken
- Which experiences were used
- Where errors or detours occurred

Algorithm 1 Trajectory Summarization

- 1: **Input:** Trajectory τ_i , correctness label c_i , ground truth y^*
 - 2: **Output:** Natural language summary s_i
 - 3: $s_i \leftarrow \text{LLM.summarize}(\tau_i, c_i, y^*)$
 - 4: **return** s_i
-

4.2.2 Stage 2: Group Advantage Extraction

Given G trajectory summaries for a query, the LLM identifies patterns distinguishing successes from failures. It outputs up to 3 operations per group: **add**, **modify**, or **delete** experiences.

Algorithm 2 Group Advantage Extraction

- 1: **Input:** Summaries $\{s_1, \dots, s_G\}$, current experiences E , ground truth y^*
 - 2: **Output:** Experience operations $\mathcal{O}_G$
 - 3: $\mathcal{O}_G \leftarrow \text{LLM.extract_advantages}(\{s_i\}, E, y^*)$
 - 4: Filter to max 3 operations: $|\mathcal{O}_G| \leq 3$
 - 5: **return** $\mathcal{O}_G$
-

4.2.3 Stage 3: Batch Consolidation

All group operations from a batch are consolidated to avoid duplication and ensure experiences are ≤ 32 words.

Algorithm 3 Batch Consolidation

- 1: **Input:** All group operations $\{\mathcal{O}_1, \dots, \mathcal{O}_B\}$, current experiences E
 - 2: **Output:** Final consolidated operations $\mathcal{O}_{\text{final}}$
 - 3: $\mathcal{O}_{\text{final}} \leftarrow \text{LLM.consolidate}(\{\mathcal{O}_j\}, E)$
 - 4: Apply operations: $E \leftarrow E \oplus \mathcal{O}_{\text{final}}$
 - 5: **return** Updated experience library E
-

4.3 Performance

On the AIME mathematical reasoning benchmark [15], Training-Free GRPO achieves:

- **AIME24:** 82.7% (vs. 80.0% vanilla GRPO, +2.7%)
- **AIME25:** 73.3% (vs. 67.9% vanilla GRPO, +5.4%)
- **Training Cost:** \$18 (vs. \$10,000+ for fine-tuning)
- **Training Samples:** 100 (vs. 10,000+ for fine-tuning)

Crucially, base model weights remain *frozen*—all improvement comes from context optimization.

5 Experience Ledger

The Experience Ledger is Zoo’s core innovation: a cryptographically verifiable, content-addressable repository of semantic optimization knowledge.

5.1 Data Structure

Each experience is a JSON object:

```
{
  "id": "exp_abc123",
  "domain": "math.algebra",
  "text": "When solving quadratics, check discriminant
          (b2-4ac) first. If negative, return 'no real
          solutions' to avoid wasted computation.",
  "confidence": 0.87,
  "examples": [
    {"input": "x2 + 2x + 5 = 0",
      "output": "No real solutions"},
    {"input": "x2 + 2x - 3 = 0",
      "output": "x = 1 or x = -3"}
  ],
  "metadata": {
    "created_at": "2025-10-17T12:00:00Z",
    "created_by": "0x742d35Cc...",
    "votes": {"upvotes": 24, "downvotes": 2},
    "usage_count": 156
  },
  "embedding": [0.123, -0.456, ...],
  "merkle_proof": "0xabc...def"
}
```

Key properties:

- **Concise:** ≤ 32 words per experience [35].
- **Strategic:** "When [context], [action]" pattern.
- **Generalizable:** Applicable to problem classes, not specific instances.
- **Verifiable:** Merkle proof ties experience to on-chain root hash.

5.2 Storage Architecture

5.2.1 IPFS: Mutable State

The current experience library is stored on IPFS [4]. Each update generates a new Content Identifier (CID). The Inference Router contract points to the latest CID.

- **Advantages:** Content-addressable, P2P replication, efficient updates.
- **Disadvantages:** CIDs change on every update; historical versions require explicit pinning.

5.2.2 Arweave: Immutable History

All historical library versions are archived on Arweave [2], a permanent storage blockchain. Each archive includes:

- Full experience library snapshot
- Merkle tree with root hash
- Timestamp and version number

This ensures *complete auditability*: anyone can reconstruct model behavior at any point in time.

5.3 Merkle Tree Verification

Experiences are organized into a Merkle tree. The root hash is stored on-chain in the Experience Registry contract:

Algorithm 4 Merkle Tree Construction

- 1: **Input:** Experience set $E = \{e_1, \dots, e_n\}$
 - 2: **Output:** Merkle root r
 - 3: Compute leaf hashes: $h_i \leftarrow \text{SHA256}(\text{JSON}(e_i))$
 - 4: Build tree: $r \leftarrow \text{MerkleTree}(\{h_i\})$
 - 5: **return** r
-

To verify an experience e_j , a user:

1. Computes $h_j = \text{SHA256}(\text{JSON}(e_j))$
2. Obtains Merkle proof π_j (path from h_j to root r)
3. Verifies $\text{VerifyProof}(r, h_j, \pi_j) = \text{True}$

If verification succeeds, e_j is guaranteed to be part of the canonical library.

5.4 DAO Governance

Experience updates undergo decentralized review:

1. **Proposal:** A contributor submits a new experience or modification. They stake KEEPER tokens (Zoo’s governance token).
2. **Voting:** Token holders vote (1 token = 1 vote). Threshold: 66% approval required.

3. **Execution:** If approved, the Experience Registry contract updates the Merkle root. The contributor receives a reward proportional to votes received.
4. **Rejection:** If rejected, the contributor loses their stake (burned or redistributed to voters).

This mechanism ensures experiences are high-quality, non-malicious, and aligned with community goals.

5.5 Experience Retrieval

Given a query q , the Semantic Context Manager retrieves the top- k most relevant experiences:

Algorithm 5 Experience Retrieval

- 1: **Input:** Query q , experience library E , number k
 - 2: **Output:** Top- k experiences E_k
 - 3: Compute query embedding: $\mathbf{v}_q \leftarrow \text{Embed}(q)$
 - 4: Compute similarities: $s_i \leftarrow \cos(\mathbf{v}_q, \mathbf{v}_{e_i})$ for all $e_i \in E$
 - 5: Sort by similarity: $E_{\text{sorted}} \leftarrow \text{sort}(E, s)$
 - 6: Select top- k : $E_k \leftarrow E_{\text{sorted}}[1 : k]$
 - 7: **return** E_k
-

Embeddings are precomputed and stored alongside experiences. Common choices: `text-embedding-ada-002` (OpenAI), `bge-large-en-v1.5` (open-source).

6 Decentralized Model Training

6.1 Federated Learning

Zoo supports federated learning [24] where data remains on contributor devices:

1. **Initialization:** Coordinator publishes initial model θ_0 and experience library E_0 on IPFS.
2. **Local Training:** Participants download θ_0 and E_0 , perform Training-Free GRPO locally with their data, produce experience updates ΔE_i .
3. **Aggregation:** Coordinator collects $\{\Delta E_i\}$, runs batch consolidation (Section 4.2.3), produces E_1 .
4. **Distribution:** Updated library E_1 published on IPFS; participants download for next round.

Unlike traditional federated learning (which aggregates gradients), Zoo aggregates *semantic experiences*. This is more efficient (natural language is compact) and interpretable (humans can audit experiences).

6.2 Privacy-Preserving Techniques

6.2.1 Differential Privacy

Experiences can be sanitized via differential privacy [11]:

$$\Pr[A(E) \in S] \leq e^\epsilon \Pr[A(E') \in S] + \delta$$

Noise is added to experience embeddings or metadata to prevent inference attacks.

6.2.2 Zero-Knowledge Proofs

For sensitive experiences, contributors can submit zero-knowledge proofs of correctness without revealing content. For example, a zkSNARK [6] proving "this experience improves validation accuracy by $\geq 5\%$ " without disclosing the experience text.

6.2.3 Secure Multi-Party Computation

Multiple parties jointly compute experience consolidation without revealing individual contributions. This is particularly useful for medical/financial domains where data is regulated.

6.3 Incentive Alignment

Contributors are rewarded via:

- **Inference Credits:** 1 hour of contribution $\approx$ 1000 free inference queries.
- **KEEPER Tokens:** Proportional to upvotes received on submitted experiences.
- **Revenue Share:** 10% of API usage fees distributed to top contributors.

This creates a virtuous cycle: high-quality contributions improve models, attract more users, generate more revenue, reward contributors.

7 Semantic Context Manager

The Semantic Context Manager maintains experience embeddings and performs retrieval.

7.1 Embedding Generation

Each experience e is converted to a dense vector $\mathbf{v}_e \in \mathbb{R}^d$ (typically $d = 768$ or $d = 1536$) via a pretrained encoder:

$$\mathbf{v}_e = \text{Encoder}(e.\text{text})$$

Common encoders:

- **OpenAI text-embedding-ada-002**: 1536-dim, high quality, proprietary.
- **BGE-large-en-v1.5** [38]: 1024-dim, open-source, competitive.
- **Sentence-BERT** [30]: 768-dim, efficient.

Embeddings are stored alongside experiences in the IPFS library.

7.2 Similarity Search

Given query embedding $\mathbf{v}_q$, compute cosine similarity with all experience embeddings:

$$\text{sim}(\mathbf{v}_q, \mathbf{v}_e) = \frac{\mathbf{v}_q \cdot \mathbf{v}_e}{\|\mathbf{v}_q\| \|\mathbf{v}_e\|}$$

For large libraries ($|E| > 10^6$), use approximate nearest neighbor (ANN) search:

- **FAISS** [19]: Facebook’s library for billion-scale vector search.
- **HNSW** [23]: Hierarchical Navigable Small World graphs.
- **ScaNN** [13]: Google’s optimized ANN.

7.3 Caching and Optimization

- **Precomputed Embeddings**: All experiences have embeddings computed offline; no real-time encoding during inference.
- **Experience Deduplication**: Before adding new experiences, check for $\text{sim}(\mathbf{v}_{\text{new}}, \mathbf{v}_e) > 0.9$. If found, merge instead of adding duplicate.
- **LRU Cache**: Frequently retrieved experiences cached in GPU memory for zero-latency access.

7.4 Quality Scoring

Each experience has a quality score:

$$Q(e) = \alpha \cdot \frac{\text{upvotes}}{\text{upvotes} + \text{downvotes}} + \beta \cdot \frac{\text{usage_count}}{\max(\text{usage_counts})} + \gamma \cdot e.\text{confidence}$$

where α, β, γ are tunable weights. Low-quality experiences are pruned when the library exceeds maximum size.

8 Inference Router

The Inference Router is a Solidity smart contract coordinating query execution.

8.1 Contract Interface

```
contract InferenceRouter {
    struct Request {
        bytes32 queryHash;
        address user;
        uint256 timestamp;
        string ipfsCID; // Experience library version
    }

    mapping(bytes32 => Request) public requests;
    mapping(address => uint256) public nodeStakes;

    function submitQuery(
        string memory query,
        string memory ipfsCID
    ) public payable returns (bytes32 requestId);

    function fulfillQuery(
        bytes32 requestId,
        string memory response,
        bytes memory proof
    ) public;

    function slashNode(address node) public;
}
```

8.2 Query Lifecycle

1. **Submission:** User calls `submitQuery(query, ipfsCID)`, pays fee in native token (ZOO).
2. **Node Selection:** Router selects GPU node via:
 - *Round-robin*: Fair distribution.
 - *Stake-weighted*: Nodes with higher stakes prioritized.
 - *Performance-based*: Nodes with lower latency prioritized.
3. **Execution:** Selected node:
 - (a) Fetches base model (frozen weights) from IPFS.

- (b) Fetches experience library at `ipfsCID`.
 - (c) Retrieves top- k relevant experiences.
 - (d) Constructs prompt: `experiences + query`.
 - (e) Runs inference.
 - (f) Optionally generates proof (zkSNARK or optimistic).
4. **Fulfillment:** Node calls `fulfillQuery(requestId, response, proof)`. Contract verifies proof (if provided) and releases payment.
 5. **Dispute:** If proof fails or response is incorrect, user can challenge. Validators re-execute query; if node is malicious, stake is slashed.

8.3 Proof Systems

8.3.1 Optimistic Proofs

Default mode: no cryptographic proof. Users can challenge within dispute window (e.g., 1 hour). Validators re-run inference; if outputs differ, slash node.

- **Pros:** Zero overhead, fast finality.
- **Cons:** Requires dispute mechanism, delayed finality.

8.3.2 zkSNARK Proofs

Node generates a succinct proof that inference was executed correctly:

$$\pi \leftarrow \text{Prove}(\text{circuit}, \text{model}, \text{input}, \text{output})$$

Contract verifies:

$$\text{Verify}(\pi, \text{output}) = \text{True}$$

- **Pros:** Instant finality, no disputes.
- **Cons:** Proving overhead ($\sim 10\times$ inference time), circuit complexity.

Current implementations: `zkML` [39], `Modulus Labs EZKL` [12].

9 GPU Compute Nodes

9.1 Node Architecture

Each GPU node runs:

- **Base Model Loader:** Downloads frozen model weights from IPFS/HuggingFace.

- **Experience Injector:** Fetches experience library, retrieves relevant experiences, constructs prompt.
- **Inference Engine:** HuggingFace Transformers, vLLM [21], TGI [17].
- **Proof Generator:** Optional zkSNARK prover.
- **RPC Interface:** Listens for queries from Inference Router.

9.2 Supported Models

- **Qwen Series:** Qwen3-4B/7B/14B/32B/72B [27]
- **LLaMA Series:** LLaMA 2/3/3.1/3.2/3.3 (all sizes) [36]
- **DeepSeek:** V2, V2.5, V3 [5]
- **Mistral/Mixtral:** Including MoE variants [18]
- **Multimodal:** Qwen3-Omni (vision + audio) [9]

9.3 Resource Requirements

Model	Parameters	VRAM (FP16)	VRAM (4-bit)
Qwen3-4B	4B	8 GB	3 GB
Qwen3-7B	7B	14 GB	5 GB
Qwen3-14B	14B	28 GB	9 GB
Qwen3-32B	32B	64 GB	18 GB
Qwen3-72B	72B	144 GB	38 GB
DeepSeek-V3	671B	1342 GB	350 GB

9.4 Multi-GPU Inference

For large models ($> 70\text{B}$ parameters), Zoo supports:

- **Tensor Parallelism:** Split individual layers across GPUs (via Megatron-LM [33]).
- **Pipeline Parallelism:** Split model depth across GPUs (via DeepSpeed [29]).
- **Sequence Parallelism:** Split sequence length (for very long contexts).

10 Economic Model

10.1 Token Utility: KEEPER

Zoo’s native token, **KEEPER**, serves three functions:

1. **Governance:** 1 KEEPER = 1 vote on experience proposals, treasury allocations, protocol upgrades.
2. **Staking:** GPU nodes must stake KEEPER as collateral. Malicious behavior results in slashing.
3. **Fee Payment:** Users pay for inference queries in KEEPER (or wrapped native tokens).

10.2 Contribution-Based Rewards

Contributors earn KEEPER via:

10.2.1 Data Contribution

Users who submit datasets for Training-Free GRPO receive:

$$\text{KEEPER}_{\text{data}} = \alpha \cdot |D| \cdot Q(D)$$

where $|D|$ is dataset size and $Q(D)$ is quality score (based on validation accuracy improvement).

10.2.2 Experience Submission

Users who submit experiences (post-DAO approval) receive:

$$\text{KEEPER}_{\text{exp}} = \beta \cdot \left(\frac{\text{upvotes}}{\text{upvotes} + \text{downvotes}} \right)^2 \cdot \log(1 + \text{usage_count})$$

This rewards both initial quality (voting) and long-term utility (usage).

10.2.3 Inference Provision

GPU nodes earn:

$$\text{KEEPER}_{\text{inference}} = \gamma \cdot \text{num_queries} \cdot \left(1 + \frac{\text{stake}}{\text{total_stake}} \right)$$

Higher stake = priority in query routing = more earnings.

10.3 Fee Structure

Model Size	Fee (KEEPER)	USD Equiv.
Qwen3-4B	0.01	\$0.005
Qwen3-7B	0.02	\$0.010
Qwen3-32B	0.05	\$0.025
Qwen3-72B	0.10	\$0.050
DeepSeek-V3	0.50	\$0.250

10% of fees burned (deflationary), 40% to GPU nodes, 40% to experience contributors, 10% to treasury.

10.4 Anti-Sybil Mechanisms

To prevent fake contributions:

- **Proof of Personhood:** Integration with WorldCoin [37] or BrightID [8].
- **Stake Requirements:** Minimum 100 KEEPER stake to submit experiences.
- **Reputation System:** Contributors build reputation over time; low-reputation users face higher scrutiny.

11 Security and Byzantine Robustness

11.1 Threat Model

11.1.1 Malicious GPU Nodes

Nodes may:

- Return incorrect outputs (to save computation).
- Inject backdoor experiences (to manipulate model behavior).
- Collude to control query routing.

11.1.2 Malicious Contributors

Contributors may:

- Submit toxic/biased experiences.
- Sybil attack to upvote low-quality experiences.
- Poison training data.

11.1.3 Network-Level Attacks

- **DDoS**: Flood Inference Router with junk queries.
- **Eclipse**: Isolate honest nodes from network.
- **Censorship**: Prevent specific experiences from propagating.

11.2 Defenses

11.2.1 Proof Systems

- **Optimistic**: Fast but requires dispute mechanism.
- **zkSNARK**: Slow but provides instant finality.
- **Hybrid**: Optimistic by default, zkSNARK for high-value queries.

11.2.2 DAO Governance

All experience updates require 66% DAO approval. This prevents single-actor manipulation.

11.2.3 Slashing

Nodes caught returning incorrect outputs lose staked KEEPER. Slashing conditions:

1. Proof verification fails.
2. Output deviates from majority (if ≥ 3 nodes execute same query).
3. Node is offline during assigned query.

11.2.4 Rate Limiting

Inference Router enforces per-user rate limits (e.g., 100 queries/hour for free tier). This mitigates DDoS.

11.2.5 Redundancy

Critical queries can be executed by $k \geq 3$ nodes. Outputs are compared via majority voting. If nodes disagree, all are slashed except the majority.

11.3 Cryptographic Primitives

- **Merkle Trees**: Verify experience inclusion without revealing full library.
- **Digital Signatures**: All on-chain transactions signed with ECDSA/EdDSA.
- **Post-Quantum**: Lux layer provides CRYSTALS-Dilithium resistance.

12 Comparison with Centralized Platforms

Aspect	Centralized (OpenAI/Anthropic)	Zoo Network
Ownership	Corporate	Community (DAO)
Transparency	Proprietary black box	Fully auditable via Arweave
Governance	Internal decisions	Token-based voting
Fine-tuning Cost	\$10,000+ per task	\$18 per task (Training-Free GRPO)
Data Privacy	Centralized logs	Federated + ZK proofs
Vendor Lock-in	High (API dependencies)	None (self-hostable)
Censorship Resistance	Single point of failure	Decentralized nodes
Interpretability	None (weight updates)	Human-readable experiences
Composability	Limited (API only)	Full (open experiences)
Economic Model	Subscription	Contribution-based rewards

12.1 Performance Comparison

Method	AIME24 (%)	AIME25 (%)	Cost (USD)
GPT-4 Turbo (OpenAI)	74.3	68.1	\$10 per 1M tokens
Claude 3.5 Sonnet	78.9	71.2	\$15 per 1M tokens
Fine-tuned Qwen3-32B	80.0	67.9	\$10,000 setup
Zoo (Training-Free GRPO)	82.7	73.3	\$18 setup + \$0.02/query

Zoo achieves state-of-the-art performance at 99.8% cost reduction.

12.2 Decentralization Metrics

- **Node Count:** Target ≥ 100 GPU nodes across 20+ geographic regions.
- **Token Distribution:** No single entity holds $> 5\%$ of KEEPER.
- **Governance Participation:** Target $\geq 30\%$ turnout on major proposals.
- **Nakamoto Coefficient:** Minimum number of nodes to control 51% of compute. Target ≥ 20 .

13 Future Work

13.1 Technical Enhancements

1. **zk-SNARK Integration:** Deploy efficient zkML circuits for instant proof verification.
2. **Multi-Agent Coordination:** Enable agents to share experiences and collaborate on complex tasks.
3. **Automated Hyperparameter Tuning:** Optimize group size, roll-out temperature, and retrieval parameters via meta-learning.
4. **Cross-Chain Bridges:** Enable Zoo experiences to be used on other chains (Ethereum, Solana, Cosmos).

13.2 Research Directions

1. **Meta-Learning Experiences:** Can experiences guide the generation of new experiences? (Self-improving curation)
2. **Private Experiences:** Use homomorphic encryption to enable private inference with encrypted experiences.
3. **Experience Markets:** Trade high-value domain-specific experiences (e.g., medical, legal) with royalties.
4. **Adversarial Robustness:** Study how malicious experiences can manipulate model behavior and design defenses.

13.3 Ecosystem Development

1. **Developer Tools:** SDKs for Python, TypeScript, Rust to interact with Zoo API.
2. **Model Zoo:** Curated collection of fine-tuned models with public experience libraries.
3. **Bounties:** Incentivize contributions to underrepresented domains (low-resource languages, specialized sciences).
4. **Partnerships:** Integrate with existing AI platforms (HuggingFace, LangChain, LlamaIndex).

14 Conclusion

Zoo Network represents a paradigm shift in artificial intelligence infrastructure. By combining decentralized blockchain coordination, semantic optimization via Training-Free GRPO, and community governance, we address the fundamental limitations of centralized AI platforms: opacity, lock-in, and cost barriers.

Our key contributions are:

- **Layered Architecture:** Clear separation of concerns across Lux (consensus), Hanzo (compute), and Zoo (AI specialization).
- **Experience Ledger:** Cryptographically verifiable, human-readable repository of optimization knowledge—replacing opaque weight updates.
- **Training-Free GRPO:** 99.8% cost reduction while achieving state-of-the-art performance on mathematical reasoning tasks.
- **Economic Incentives:** Contributors earn inference credits, governance rights, and revenue shares—aligning incentives for long-term ecosystem growth.
- **Privacy Preservation:** Federated learning and optional zero-knowledge proofs enable sensitive use cases without data centralization.

As foundation models become more powerful, the question is not whether AI will be decentralized, but *how* and *when*. Zoo Network provides a concrete, deployable answer: a fully-functional L2 stack combining state-of-the-art machine learning with blockchain’s transparency, composability, and censorship resistance.

We invite researchers, developers, and communities to join us in building the future of decentralized AI. The code is open-source, the data is community-owned, and the governance is democratic. Together, we can ensure AI serves humanity—not corporate interests.

Acknowledgments

Zoo Network is developed by Zoo Labs Foundation Inc, a 501(c)(3) non-profit organization. We thank the Hanzo Network team for providing the base compute infrastructure, the Lux Blockchain team for consensus primitives, and the open-source community for tools and libraries. Special thanks to Tencent for releasing the Training-Free GRPO paper and reference implementation.

References

- [1] Akash Network. Akash: Decentralized Cloud Compute Marketplace. Technical report, 2021.
- [2] Sam Williams and Viktor Tron. Arweave: A Protocol for Economically Sustainable Information Permanence. Technical report, 2018.
- [3] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? In *FAccT*, 2021.
- [4] Juan Benet. IPFS - Content Addressed, Versioned, P2P File System. *arXiv:1407.3561*, 2014.
- [5] DeepSeek-AI. DeepSeek-V3: Technical Report. Technical report, 2024.
- [6] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. Succinct Non-Interactive Arguments via Linear Interactive Proofs. In *TCC*, 2013.
- [7] Bittensor Foundation. Bittensor: A Peer-to-Peer Intelligence Market. Technical report, 2021.
- [8] BrightID. BrightID: A Proof of Uniqueness Protocol. Technical report, 2021.
- [9] Yunfei Chu et al. Qwen2-Audio: Technical Report. *arXiv:2407.10759*, 2024.
- [10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *NeurIPS*, 2023.
- [11] Cynthia Dwork and Aaron Roth. The Algorithmic Foundations of Differential Privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3-4):211-407, 2014.
- [12] Modulus Labs. EZKL: Easy Zero-Knowledge Machine Learning. <https://github.com/zkonduit/ezkl>, 2023.
- [13] Ruiqi Guo et al. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *ICML*, 2020.
- [14] Hanzo Network. Hanzo: Decentralized Compute Infrastructure for AI. Technical report, 2025.
- [15] Dan Hendrycks et al. Measuring Mathematical Problem Solving With the MATH Dataset. In *NeurIPS*, 2021.

- [16] Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.
- [17] HuggingFace. Text Generation Inference. <https://github.com/huggingface/text-generation-inference>, 2023.
- [18] Albert Q. Jiang et al. Mistral 7B. *arXiv:2310.06825*, 2023.
- [19] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535-547, 2019.
- [20] Hyesung Kim, Jihong Park, Mehdi Bennis, and Seong-Lyun Kim. Blockchain On-Device Federated Learning. *IEEE Communications Letters*, 24(6):1279-1283, 2019.
- [21] Woosuk Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.
- [22] Lux Network. Lux: Multi-Consensus Blockchain with Post-Quantum Cryptography. Technical report, 2025.
- [23] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824-836, 2018.
- [24] H. Brendan McMahan et al. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *AISTATS*, 2017.
- [25] Ocean Protocol Foundation. Ocean Protocol: A Decentralized Data Exchange Protocol. Technical report, 2021.
- [26] Long Ouyang et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.
- [27] Qwen Team. Qwen3 Technical Report. Technical report, 2024.
- [28] Rafael Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *NeurIPS*, 2023.
- [29] Jeff Rasley et al. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *KDD*, 2020.
- [30] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP-IJCNLP*, 2019.
- [31] Team Rocket et al. Scalable and Probabilistic Leaderless BFT Consensus through Metastability. *arXiv:1906.08936*, 2020.

- [32] Zhihong Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.
- [33] Mohammad Shoeybi et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv:1909.08053*, 2019.
- [34] SingularityNET Foundation. SingularityNET: A Decentralized, Open Market and Network for AIs. Technical report, 2021.
- [35] Tencent youtu-agent Team. Training-Free Group Relative Policy Optimization. *arXiv:2510.08191*, 2025.
- [36] Hugo Touvron et al. LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971*, 2023.
- [37] Worldcoin Foundation. Worldcoin: A Privacy-Preserving Proof-of-Personhood Protocol. Technical report, 2023.
- [38] Shitao Xiao et al. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv:2309.07597*, 2023.
- [39] zkML Community. Zero-Knowledge Machine Learning. <https://github.com/zkml-community>, 2023.
- [40] Zoo Labs Foundation. Hamiltonian Large Language Models: Training-Free Semantic Optimization. Technical report, 2025.

A Experience Library Examples

A.1 Mathematical Reasoning

- [G0]. When solving geometry problems with intersections, validate solutions lie within bounded regions, not extensions, to avoid extraneous answers.
- [G1]. For expected extreme statistics in combinatorial problems, use direct enumeration for small sizes.
- [G10]. When using mathematical invariants to prove impossibility, always validate against known achievable states or small cases.
- [G21]. For complex polynomials with real parameters, separate real/imaginary parts to find when real roots exist.

A.2 Code Generation

- [C0]. Before implementing complex algorithms, write unit tests for edge cases (empty input, single element, maximum size).
- [C5]. When optimizing nested loops, consider whether inner loop can be vectorized or replaced with hash map lookup.
- [C12]. For recursive functions, always define base case first and verify termination condition.

A.3 Creative Writing

- [W0]. When developing character arcs, establish clear motivation in first act to justify later decisions.
- [W3]. In descriptive passages, prefer specific sensory details over generic adjectives.
- [W7]. For dialogue, ensure each character has distinct speech patterns reflecting background/personality.

B Deployment Guide

B.1 Running a GPU Node

B.1.1 Prerequisites

- NVIDIA GPU with ≥ 16 GB VRAM
- CUDA 12.1+
- Docker 24.0+
- Staked KEEPER tokens (≥ 1000)

B.1.2 Setup

```
# Clone Zoo node repository
git clone https://github.com/zoo-labs/zoo-node
cd zoo-node
```

```
# Configure environment
```

```

cp .env.example .env
# Edit .env: set RPC_URL, STAKING_KEY, IPFS_GATEWAY

# Build Docker image
docker build -t zoo-node:latest .

# Run node
docker run -d \
  --gpus all \
  -p 8080:8080 \
  -v /data:/data \
  --env-file .env \
  zoo-node:latest

```

B.1.3 Monitoring

```

# Check node status
curl http://localhost:8080/status

# View logs
docker logs -f <container_id>

# Check earnings
curl http://localhost:8080/earnings

```

B.2 Submitting Experiences

B.2.1 Via Python SDK

```

from zoo_sdk import ZooClient, Experience

client = ZooClient(rpc_url="https://rpc.zoo.ngo")

# Create experience
exp = Experience(
    domain="math.algebra",
    text="When solving quadratics, check discriminant
        first. If negative, no real solutions.",
    examples=[
        {"input": "x2 + 2x + 5 = 0",
         "output": "No real solutions"}
    ]
)

# Submit for DAO review

```

```

tx_hash = client.submit_experience(
    experience=exp,
    stake_amount=100 # KEEPER tokens
)

print(f"Submitted: {tx_hash}")

```

B.2.2 Via Web UI

1. Navigate to <https://app.zoo.ngo>
2. Connect wallet (MetaMask/WalletConnect)
3. Go to "Contribute" → "Submit Experience"
4. Fill form: domain, text, examples
5. Approve stake (100 KEEPER)
6. Submit for review

C Governance Parameters

Parameter	Value	Description
Voting Period	7 days	Duration for proposal voting
Quorum	10%	Min. participation required
Approval Threshold	66%	Required yes votes
Min. Stake (Proposal)	1000 KEEPER	To create proposal
Min. Stake (Experience)	100 KEEPER	To submit experience
Slashing Penalty	50%	Stake lost if malicious
Experience Max Size	32 words	Per experience
Library Max Size	1000 experiences	Total capacity